

ECE243 Final Report

Adrian Yip and Rikuto Ide

University of Toronto

1 Overview

We have developed a Pokemon RPG inspired game that makes use of the PS/2 keyboard and audio device interrupts to simulate battling, map traversal, and monster catching. The VGA displays monster and character movements, UI/UX changes, and other relevant game animations. This report documents and provides a high-level summary of the architecture, logic, and design choices behind this project.

2 System Block Diagram

Fig. 1 is our current block diagram, showing a high level overview of our project. This diagram has gone through several iterations, where our ideas and work from week 1 and 2 have been preserved and improved upon.

Sections 3 through 7 discuss these systems in more detail while Sections 8 through 10 discuss additional information about the project, including design choices, future implementation, and unit testing.

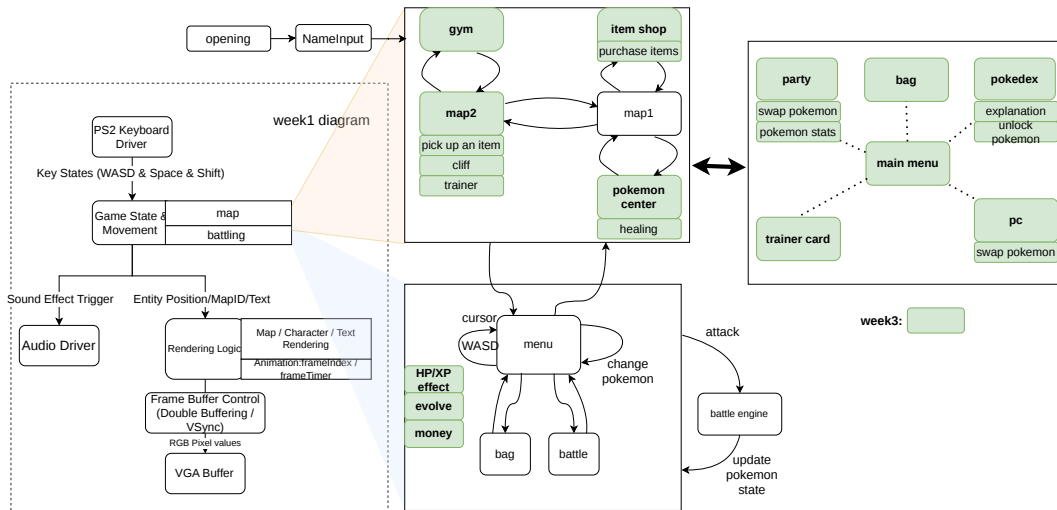


Fig. 1: Week 3 diagram.

3 Animation and Graphics

3.1 Rendering Targets and Constraints

This project's graphics system uses a VGA output with the CPU-only renderer on the NiosV. All graphics are produced by writing directly into a memory mapped pixel buffer, presented using double buffering. The VGA pixel framebuffer has a fixed resolution of 320 x 240 (`SCREEN_WIDTH`, `SCREEN_HEIGHT`) where pixels are stored as 16-bit RGB565 values. Pixels are written using the following address calculation in order to place sprites from the desired XY coordinates onto the VGA display: $(y \ll 10) + (x \ll 1)$.

With the absence of GPU acceleration, all special visuals are done with deterministic and lightweight effects, rather than computationally heavy rendering methods. All of the drawing operations are done using loops that write into the back buffer, where `wait_for_vsync()` is called regularly to update the display.

3.2 Frame Buffering

Graphics are rendered to a VGA pixel double buffer in `FRAMEBUFFER_0` and `FRAMEBUFFER_1`, which are initialized with `init_graphics()`. All drawing is performed to the back buffer, where `wait_for_vsync()` is called to synchronize, swap, and update the stored buffer pointers so future frames are drawn off screen before presented.

During initialization, both buffers are cleared. This is done to prevent any garbage pixels (from previously used memory) from appearing after the first swap to ensure we only see the desired display.

Without double buffering, drawing directly onto the displayed buffer would cause tearing and flickering, where the screen would have partially updated frames. This unintended behavior happens because the VGA would be reading the framebuffer while the CPU is still writing into it.

Double buffering avoids this issue by only presenting complete frames at vsync boundaries, at the expense of memory usage. A frame buffer stores a full screen of 320x240 RGB565 pixels, requiring 153,600 bytes of memory. Using two for double buffering consumes a larger fraction of the available RAM and SDRAM compared to a single buffer, but eliminates tearing by ensuring the VGA avoids scanning a buffer a CPU is currently modifying.

3.3 Asset Pipeline

Art assets were obtained through sprite resource websites [1], databases [2], GitHub repositories [3], and various art sharing platforms [4], [5], [6], [7]. These assets were then modified using online sprite editors [8] and automatically converted from RGBA to RGB565 image pixels into arrays that match the VGA frame buffer's format using python code. `0xF81F` (`TRANSPARENT_COLOUR`) was used as a consistent transparency convention, where during rendering, pixels equal to this key would be skipped so the background shows through.

3.4 Sprite Systems and Timings

Sprites are drawn by reading the converted sprite arrays and copying non-transparent pixels into the current back buffer, and animations are produced by advancing a frame index at a desired tick rate. Static pixel arrays are used for individual frames (`const unsigned short frame[]`) and frame pointers tables (`const unsigned short* const frames[]`) advance the individual frames for animated sprites. `graphics.c` contains the rendering methods used in the project, frequently calling `draw_sprite_any(sprite, width, height, x, y, TRANSPARENT_COLOUR)` to display sprites at the desired location.

Pokemon sprites have compiled front and back sprite arrays and associated width/height macros. When a battle is initialized, `initPokemonBattleSprite()` is called based on the Pokemon's species ID to lookup either the front or back asset and populate a `StaticSprite` with `pixels`, `width`, `height`, and the desired position.. The program then uses `initPokemonBackBattleSprite()` and `initPokemonFrontBattleSprite()` to place the desired sprites at fixed battle coordinates, outlined in Table 1.

Table 1: Battle Sprite Coordinates (defined by the top left corner of the sprite)

Sprite	Definition	X/Y Coordinate
User's Pokémon (Back Sprite) X	<code>POKEMON_BATTLE_BACK_X</code>	32
User's Pokémon (Back Sprite) Y	<code>POKEMON_BATTLE_BACK_Y</code>	77
Opponent Pokémon (Front Sprite) X	<code>POKEMON_BATTLE_FRONT_X</code>	190
Opponent Pokémon (Front Sprite) Y	<code>POKEMON_BATTLE_FRONT_Y</code>	17

3.5 Effects

To improve visual feedback without requiring additional art assets or animations, the project implements several software based post processing and motion effects, summarized in Table 2. These effects are applied during drawing through transformations or small deterministic offsets before being placed into the back buffer.

Table 2: Sprite Rendering Effects and Use Cases

Effect	Usage	Description
Outline	Visual Feedback for Selecting Shop Items or Attacks	A rectangular outline is applied around a chosen area, where the colour is also chosen (<code>draw_rect_outline()</code>).
Shake	Hit Reaction and Damage Feedback	A pre-defined horizontal offset pattern (<code>dx_pattern</code>) is applied, where the sprite is drawn at $x + dx$ (<code>draw_sprite_any_shake()</code>).
Bob	Selected Pokemon Feedback	A pre-defined vertical offset pattern (<code>dy_pattern</code>) is applied, where the sprite is drawn at $y + dy$ (<code>draw_sprite_any_bob()</code>).
Pulse	Selected UI Item Feedback	A repeated brightness delta pattern (<code>shade_pattern</code>) is applied using <code>shade_565()</code> (<code>draw_sprite_any_shade_pulse()</code>).
Greyscale	Visual Indicator for Unavailable Pokemon	Uses <code>greyscale_565()</code> to convert a sprite to greyscale (<code>draw_sprite_any_greyscale()</code>).

3.6 UI/UX Readability

UI screens are created using fixed pixel coordinates of different elements (menus, HP bars, status icons, prompts) and are drawn over background sprites to communicate the current state of the game and available actions. The main menu, shown in Fig. 2 acts as the central hub of user activity, where they are able to view/switch their Pokemon, check their Pokedex/items, and view their Trainer Card stats.



Fig. 2: The WASD keys are used to navigate between the different menus, where users can press two sequential directions at once to navigate to the diagonal icons as well.

Text is rendered using bitmap fonts [9] in two styles (`FONT_8X8` and `FONT_5X9`) which are drawn with `draw_string_f()`. Textbox sprites are used to simulate dialogue and battle prompts, with both immediate (`draw_textbox_instant_text()`) and animated (`draw_textbox_animated_text()`) methods.

The battling engine was structured to output events as ordered messages through the `BattleState`, containing `messages[BATTLE_MSG_MAX]` [96] and additional information such as effectiveness indicators and HP values. This was done to allow for display messages to display sequentially or have them synchronized to a specific message index (displaying "used move", then animate a shaking sprite, decrementing HP, then displaying "super effective" and victory text). As these messages advance, context is stored in the battle messages as well (`displayPlayerIndex[]` and `displayEnemyIndex[]`) as a defensive mechanism to prevent the screen from desyncing from the shown messages.

3.7 Complex Animations

The catching sequence inside battle state makes use of two different trajectory models to emulate a "throwing" effect for catching and sending out Pokemon. In both cases, a Pokeball's sprite frame is advanced independently (using `pokeballThrowFrames[]`), while the position of the sprite varies and is produced by a mathematical formula every tick.

3.7.1 Projectile Motion (Battle Entry Throwing)

During `GAME_STATE_POKEBALL_THROW`, the Pokeball follows a trajectory that mimics projectile motion with constant acceleration to throw your own Pokemon onto the battlefield. The Pokeball is "thrown", appearing to be traveling in an arc shape and moving left to right. The trajectory is modeled as:

$$y(t) = y_{0,\text{pokeball}} + v_{y,\text{pokeball}} t + \frac{1}{2} g_{\text{pokeball}} t^2, \quad x(t) = x_{0,\text{pokeball}} + v_{x,\text{pokeball}} t$$

where

- **Peak time** ($T_{\text{peak}} = 18$) determines the number of ticks for the Pokeball to reach the max height.
- **Gravity** ($g = 0.35$) controls the arc shape, with a lower value producing a flatter trajectory so the ball.
- Both values are **adjusted for animation**, rather than representing real world conditions.

The vertical distance from peak to landing is

$$\Delta y_{\text{pokeball}} = y_{\text{land,pokeball}} - y_{\text{peak,pokeball}},$$

and the total flight time is

$$T_{\text{land,pokeball}} = T_{\text{peak,pokeball}} + \sqrt{\frac{2\Delta y_{\text{pokeball}}}{g_{\text{pokeball}}}}.$$

The horizontal velocity is then chosen to satisfy:

$$x(T_{\text{land,pokeball}}) = x_{\text{land,pokeball}}.$$

3.7.2 Quadratic Bézier Curve (Throwing Pokeball at Opponent) [10]

During `GAME.STATE.POKEBALL_CATCH`, the Pokeball throw toward the opponent's Pokemon uses a quadratic Bézier curve, providing a controllable arc between two anchored endpoints. This trajectory is modeled as:

$$\mathbf{P}_{\text{pokeball}}(u) = (1-u)^2\mathbf{P}_{0,\text{pokeball}} + 2(1-u)u\mathbf{P}_{1,\text{pokeball}} + u^2\mathbf{P}_{2,\text{pokeball}}, \quad u \in [0, 1]$$

where

- $\mathbf{P}_{0,\text{pokeball}}$ is the start point near the user's sprite.
- $\mathbf{P}_{1,\text{pokeball}}$ is a control parameter above the midpoint to shape the arc (offset by 70 pixels).
- $\mathbf{P}_{2,\text{pokeball}}$ is the end point near the enemy sprite.
- $u \in [0, 1]$ is the Bézier parameter that controls the Pokeball's location along the curve.
- u increases linearly with time steps: $u = \frac{t}{t_{\max}}$, where $t_{\max} = 36$ ticks (chosen for animation timing).

4 Game Databases

4.1 Pokemon Database

This project makes use of databases as the core Pokemon data, allowing for deterministic lookups by ID. This data is split into three focused modules:

4.1.1 Pokemon Data (Game Stats)

Pokemon are represented with two layers of data: a species template and an in-memory Pokemon object.

`PokemonData` defines the species of each Pokemon and contains fixed information for the species:

- `id`, `name`
- Base Stats: `baseHp`, `baseAttack`, `baseSpAttack`, `baseDefense`, `baseSpDefense`, and `baseSpeed`
- Typing: `type1`, `type2`
- Evolution Rules: `EvolutionData`

`pokemonInBattle` defines the current Pokemon in battle in a per-Pokemon instance combining the pointer to the immutable species data `PokemonData` with the following mutable gameplay states:

- `pokemonID`
- `level` and `exp`
- Meeting Location (for caught Pokemon): `metMapId` and `metLevel`
- Scaled Stats and Modifiers to Stats: `Hp`, `Attack`, `SpAttack`, `Defense`, `SpDefense`, and `Speed`
- PP for each attack: `currentPP[4]`
- Gamestate Flags: `inBattle`, `alive`, `pendingLearnedMoveCount`, `StatusCondition` `status`, and `PokemonData *pendingEvolutionInto`
- Attack Data: `AttackData *attacks[4]` and `AttackData *pendingLearnMoves[4]`
- Typing: `type1`, `type2`

`initPokemonInBattle()` initializes the Pokemon for battle by binding `id.data` to a `PokemonData` template. Then the Pokemon seeds moves from their defined learnsets with `initMovesFromLearnset()` and scales their stats based on their current level and base stats with `scaledStatesWithLevel()`. Fig. 3 shows an example of an initialized `pokemonInBattle` instance, with scaled stats and required EXP. An example of a `pokemonInBattle` instance in a battle is shown with Fig. 4, where the instance RAYQUAZA had its moves populated.



Fig. 3: Stat Screen of a Pokemon



Fig. 4: Move Selection Screen

Battle specific data including stat stages (modifying attack/defense, etc), accuracy/evasion, `inBattle` flags, and status conditions are mutable at the resolution of each combat turn, and is kept track of.

A Pokemon's move slots (`const AttackData *attacks[4]`) all have separate PP amounts, tracked with (`currentPP[4]`). This means that the move database stays "read only" while the PP is tracked for each Pokemon instance.

Gained experience points (exp) and level-ups change the specific instance, where evolutions are checked and called with `pendingEvolutionInfo()` and applied with `applyPendingEvolution()` when the conditions set by `EvolutionData` in `PokemonData` is met, swapping templates while preserving all the relevant instance states.

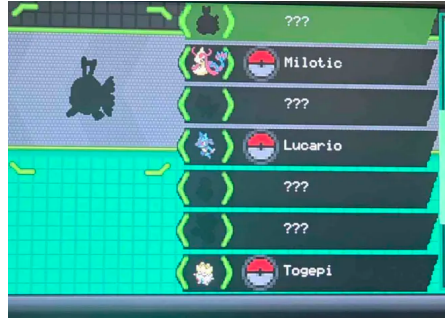
Each `pokemonInBattle` struct is stored into the PC, referenced by the user's party slots during battle.

4.1.2 Pokedex Entries

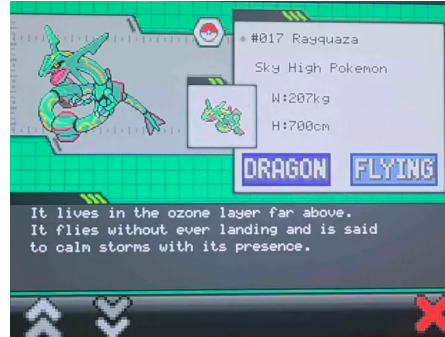
The Pokedex database system provides descriptions for each Pokemon ID and tracks player progression with a **seen** and **caught** status for each species. `PokedexEntry` stores `pokemon_id`, `category`, `height_cm`, `weight_kg`, and their corresponding Pokedex entry (`entry`).

These entries are put into `POKEDEX_DB[]`, a static table, which is read only at runtime. Populating the Pokedex is done through a linear search through `POKEDEX_DB[]` with `getPokedexEntry(int pokemon_id)` and returns a matching entry. `pokemon_id` values are consistent with sprite IDs and in `PokemonData`, enabling the Pokedex to link together with the Pokemon instances in battle and the displayed sprites.

The global arrays `g_pokedexSeen[]` and `g_pokedexCaught[]` are updated with `pokedex_mark_seen()` and `pokedex_mark_caught()` to track the user's progress in discovering Pokemon. Fig. 5 showcases the Pokedex, where Pokemon inside `g_pokedexCaught[]` have their information displayed, while Pokemon in neither `g_pokedexSeen[]` or `g_pokedexCaught[]` are shaded in black.



(a) First Page of the Pokedex



(b) Second Page of the Pokedex

Fig. 5: Pokedex Screens

4.1.3 Pokemon Storage

PC is a container that owns the Pokemon instances as an array of `mons[PC_MAX]`, where `PC_MAX = 25`. The program calls `pcInit()` at the start of the game to clear the storage and initializes new owned instances of Pokemon with `pcAdd()`. `pcGet` returns a pointer to the desired stored instance. Fig. 6 shows an image of the PC storage system, where users are able to switch and use any caught Pokemon.



Fig. 6: An example of the PC storage, where users can swap and use any caught Pokemon.

4.2 Bag, Items, and Currency

This project supports the use of items inside and outside battle (Pokeballs, healing items, and evolution items). The item subsystem defines a fixed set of item IDs along with a static database for them. The bag keeps track of the user's inventory which persists throughout the game. There is also a currency model used for shop purchases and trainer battle payouts/penalties.

ItemData is stored in a static table, `ITEM_DB[ITEM_ID_COUNT]`, which can be accessed with `getItemData()` and `getItemDescription()`.

The bag stores the user's quantities of each item with `counts[ITEM_ID_COUNT]` and the `lastUsedItem`. Item values are modified for a user's bag with `bagAdd()` and `bagRemove()`, where the quantity of each item is clamped at 999.

Items are also categories with `itemIsHealing()`, `itemIsRevive`, and `itemIsBall` for sorting with the various bag UIs. Fig. 7 shows an example user Bag, on the ITEMS page.

`int playerMoney` is tracked and initialized to `STARTING_MONEY`. Winning against trainers will yield a payout while losing will apply a payout penalty. These values are clamped to a percentage of the user's current money and are dependent on the enemy party level, using `computeTrainerPayout()` and `computeLossPenalty`.

Inside the item shop, `shopItemPrice(ItemId)` defines the prices for available items, where users are able to spend their money. Fig. 8 shows the item shop UI, showcasing the items for purchase and the corresponding costs.

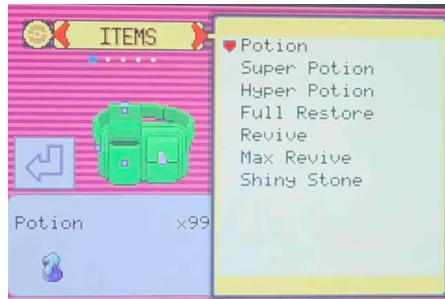


Fig. 7: An example of a user's bag in the game.

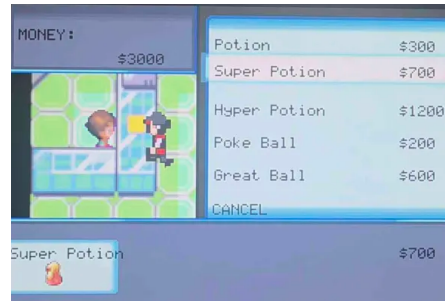


Fig. 8: The Item Shop

5 Battling Engine

5.1 Battle Flow

The battle flow is driven by `BattleState`, where each user input (`ACTION_ATTACK`, `ACTION_SWITCH`, `ACTION_ITEM`, `ACTION_RUN`) is applied through `battleApplyPlayerAction()`, resolving the user/enemy turns, updates the message queue, and check for win/lose/flee/catch. A high level overview is shown in Fig. 9.

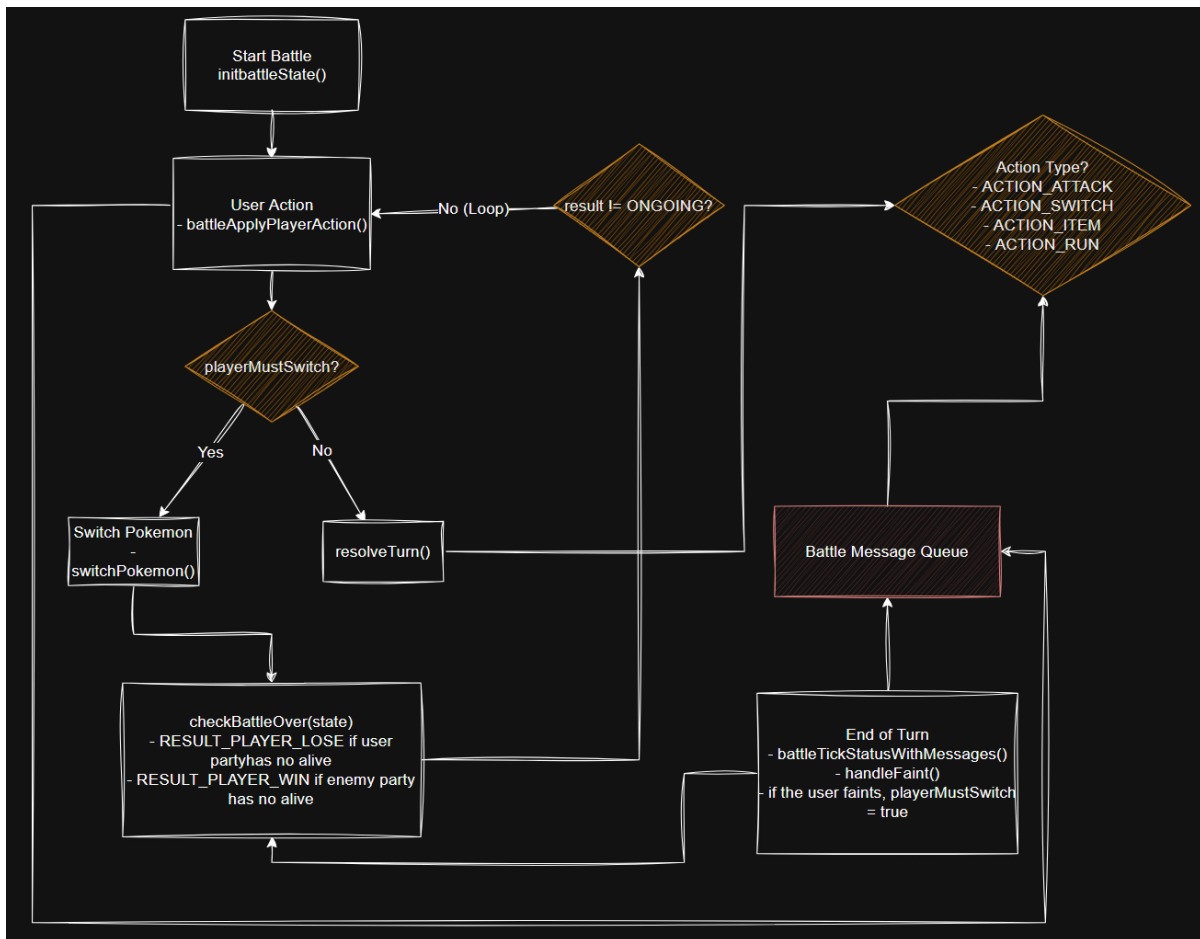


Fig. 9: A High Level Overview of the Battling Flow

5.2 Party Management

Party is a team that holds up to 6 points of instances. `initParty` is used at the start of the game to clear and reset party slots while `addPokemonToParty()` is used to append a point and `getActivePokemon()` returns the user's current Pokemon in battle or the Pokemon that will be sent out first in battle.

In battle, `switchPokemon()` changes the `activeIndex` of the user's team, after defensively checking if the new Pokemon is alive and a valid team member. After switching, any current stat modifiers are reset for both the incoming and outgoing active Pokemon through `resetStatStages()` to have consistent behavior to actual Pokemon games.

The functions `hasAlivePokemon()` and `getFirstAlivePokemon()` are used to enforce swapping when the user's current Pokemon has fainted or when the user has no more alive Pokemon on their team.

5.3 Combat Rules

5.3.1 Damage and Status Effects

When battling, damage is dealt using `dealDamage()`, which uses a similar formula to Pokemon to calculate damage [11]. Damage takes the levels, move power, appropriate offensive/defensive stats, stat modifiers, STAB, and a small random factor into account. For escaping and catching, they are only allowed when facing a wild Pokemon, not Trainer Battles.

Type chart effectiveness is present in damage calculations as well, using `typeChart[NUM_TYPES][NUM_TYPES]` and `getTypeEffectiveness()` to determine "Super Effectiveness" (2x damage), "Resistance" (0.5x damage), or "Immunity" (0x damage). Fig. 10 is shown to clarify the type chart for effectiveness for anybody unfamiliar with Pokemon games.

`useAttack()` consumes PP, then performs an accuracy check based on the attacker's accuracy and the target's evasion stage. Then a random roll is performed to determine a hit or miss. `useAttack()` also handles any stat changes or status effects applied in battle, defensively checking for immunities (eg. ice types cannot be frozen) and valid stat changes (eg. a Pokemon with +6 attack can not increase attack any higher).

If a status condition is applied to a Pokemon, `tickStatusEffect()` is applied at the end of each turn to deal damage over time (DoT) for `STATUS_POISON/STATUS_BURN` and manages `STATUS_SLEEP/STATUS_FREEZE` durations. `canActThisTurn()` is called for every attack, preventing actions for `STATUS_SLEEP`, `STATUS_PARALYSIS`, or `STATUS_FREEZE` affected Pokemon. Similar to the main Pokemon games, only the `STATUS_SLEEP` and `STATUS_FREEZE` effects can wear off naturally, which `STATUS_BURN` also lowering attack and `STATUS_PARALYSIS` lowering speed.

Defender	Normal	Fire	Water	Grass	Electric	Ice	Fighting	Poison	Ground	Flying	Psychic	Bug	Rock	Ghost	Dragon	Dark	Steel	Fairy
Attacker	Normal	Fire	Water	Grass	Electric	Ice	Fighting	Poison	Ground	Flying	Psychic	Bug	Rock	Ghost	Dragon	Dark	Steel	Fairy
Normal													1/2	0			1/2	
Fire			1/2	1/2	2		2					2	1/2		1/2		2	
Water		2		1/2	1/2				2				2		1/2			
Grass		1/2	2	1/2				1/2	2	1/2		1/2	2		1/2		1/2	
Electric			2	1/2	1/2			0	2						1/2			
Ice		1/2	1/2	2		1/2		2	2						2		1/2	
Fighting	2					2		1/2		1/2	1/2	2	2	0		2	2	1/2
Poison				2				1/2	1/2			1/2	1/2			0	2	
Ground		2		1/2	2			2		0		1/2	2					2
Flying				2	1/2		2		2			2	1/2					1/2
Psychic							2	2			1/2				0	1/2		
Bug		1/2		2			1/2	1/2		1/2	2			1/2		2	1/2	1/2
Rock		2				2	1/2	1/2	2		2						1/2	
Ghost	0										2			2		1/2		
Dragon															2	1/2	0	
Dark							1/2			2				2		1/2		1/2
Steel		1/2	1/2		1/2	2						2					1/2	2
Fairy		1/2				2	1/2								2	2	1/2	

Fig. 10: Type Chart Effectiveness for Pokemon [12]

5.3.2 Catching

If a user decides to catch a wild Pokemon, `attemptCatchWithBall()` is called, computing the catch success with a deterministic luck modifier. This luck modifier is computed with the type of Pokeball, remaining HP ratio, and status bonuses, shown in Tables 3-6.

Table 3: Wild Pokémon HP Factor

Wild Pokemon HP State	Formula	Multiplier
General Case	$\text{hpFactor} = 1.0 + 2.0 \left(1.0 - \frac{\text{curHP}}{\text{maxHP}} \right)$	1.0 at full HP to 3.0 at 0 HP

Table 4: Status Condition Bonus

Status	Multiplier
STATUS_NONE	1.0
STATUS_SLEEP	2.0
STATUS_FREEZE	2.0
STATUS_PARALYSIS	1.5
STATUS_BURN	1.5
STATUS_POISON	1.5

Table 5: Pokeball Type Modifier

Ball Type	Multiplier
POKEBALL_POKE	1.0
POKEBALL_GREAT	1.5
POKEBALL_ULTRA	2.0
POKEBALL_PREMIER	1.0
POKEBALL_MASTER	255.0 (guaranteed)

Table 6: Combined Catch Strength

Computation	Formula
Catch Strength	$\text{catchChance} = \text{hpFactor} \cdot \text{statusBonus} \cdot \text{ballModifier}$
Success Check	$(\text{rand}() \% 256) < \lfloor \text{catchChance} \times 50 \rfloor$

5.3.3 Escaping

If the user attempts to "Run" from a wild battle, `attemptFlee()` is called and compares the speeds of the two active Pokemon. Calculations are shown in Tables 7 and 8.

Table 7: Escape Formulas

Condition	Result
$\text{fleeingSpd} \geq \text{opponentSpd}$	Guaranteed escape
$\text{fleeingSpd} < \text{opponentSpd}$	Escape probability calculation used

Table 8: Escape Probability (if Slower)

Computation	Formula
Flee Chance	$\text{fleeChance} = \frac{\text{fleeingSpd} \times 128}{\text{opponentSpd}} + 30$
Success Check	$(\text{rand}() \% 256) < \text{fleeChance}$

5.4 Battle Messages

Battle feedback is given to the player through a queued message system (`messages[BATTLE_MSG_MAX][96]`). `messages[]` is built inside the `battleLoop.c` after every resolved action (eg. "used move", "super effective", "missed", "fainted"). Data is also stored alongside the queued message, for the UI to synchronize with visual effects.

5.5 Post Battle Logic

If the opposing Pokemon faints or is caught, `awardExpForDefeat()` is called and awards exp to the user's active pokemon. `experienceGained()` computed the awarded exp, `gainExp()` increments EXP and calls `levelUp()` and `checkLevelUpMoves()` when a Pokemon meets the required EXP to level up.

The EXP gained formula is calculated using $EXP_{gained} = 50 * (opponentlevel)$, which differs from traditional Pokemon games and was implemented intentionally to make progression faster for demoing our project. We decided to stray away from the traditional EXP formulas to keep leveling system simple, deterministic, and easy to modify, while still rewarding defeating higher leveled opponents with proportionally more EXP. This design choice also makes progression behavior more predictable during short play sessions.

The traditional Pokemon EXP formula is shown below:

5.5.1 Reference EXP Formula for Generation VI [13]

$$\Delta EXP = \left(\frac{bL}{7} \right) a \left(\frac{1}{s} \right) t e v f p$$

Variable Definitions

- b : base experience yield of the defeated Pokemon's species
- L : level of the fainted Pokemon
- s : EXP split factor (depends on generation, participation, and Exp. Share rules)
- a : trainer bonus (1.5 if the fainted Pokemon is trainer owned, otherwise 1)
- e : Lucky Egg multiplier (1.5 if held, otherwise 1)
- t : outsider/original trainer multiplier (1 if original trainer; 1.5 if outsider)
- p : EXP Point Power / boost multiplier (varies by game/power; 1 if none)
- f : affection/friendship bonus multiplier (when applicable; otherwise 1)
- v : "able to evolve but hasn't" multiplier (otherwise 1)

If a Pokemon meets evolution requirements on a level up, the evolution is stored with `pendingEvolutionInto()`, where the UI and game state will change and handle evolution after a battle has been completed.

6 keyboard/audio

The keyboard system reads scan codes from the PS/2 keyboard and converts them into simple inputs used in the game, such as WASD, arrow keys, Space, Tab, and Enter. The function `update_keyboard()` detects key press and release. Movement keys are kept as states, while text input is stored in a queue, so it can be used for player name input and menu operations. This allows movement, selection, menu access, and text input to be handled in a single input system.

The audio system mixes background music (BGM), sound effects, and foot-step noise, and sends them to the audio output to create sound effects like in a Pokémon game. First, we implemented Python code to convert MP3 files into arrays and used those arrays as audio data in the system [14] [15]. Functions like `play_bgm()` and `play_sfx()` switch the audio data, and `audio_update()` mixes multiple sounds and outputs them to the left and right channels. It also uses interrupts to continuously supply audio, allowing sound to match actions such as moving between maps, battles, and menu operations.

7 maps

The map system uses a 20×15 tile-based 2D array to place terrain, buildings, indoor backgrounds, and NPCs, and render the current area on the screen. Functions like `load_world_map()` and `load_map_preset()` switch between different layouts, such as outdoor maps and indoor areas like the Pokémon Center, and apply decorations and building placements for each map. It also checks the player's position and movement to handle transitions between areas, such as entering buildings or moving to adjacent maps.

In addition, the main character's bounding box was also implemented for collision detection. Using the bounding box, the system can detect collisions with buildings and NPCs, and also supports map gimmicks such as cliffs that can only be passed in one direction.

8 Timing and Rates

Our project uses a frame dependent model to update the display, with a timing baseline of **20 frames per second** (≈ 50 ms per frame). This provides deterministic timing for animations and UI drawing, since the ingame behaviors such as sprite frame delays, animated text speed, visual feedback, and transitions are defined and tuned in discrete frame units rather than real time delays.

Doing so allowed our game to have visuals that remain consistent throughout different states and can be synchronized predictably with gameplay events (such as message queues or effects).

9 Design Choices

9.1 Timed and Deterministic Message Output for Battles

Battle feedback is stored in the message queue, (`messages[BATTLE_MSG_MAX]` [96]), rather than being printed directly or dynamically allocated. This was done intentionally to allow for the battle text to sync with any effects we wanted to add for visual feedback.

9.2 Colour Key Transparency

Sprites use a single transparency key (`TRANSPARENT_COLOUR = 0xF81F`) so that background pixels can be skipped during drawing. We decided to use a colour key over transparent pixels as **colour keying** is a common 2D technique for game developers. [16]

9.3 Separation of Species Data and In-Memory Instances

Static Pokémon templates (base stats, types, evolutions, move definitions, type chart) are stored in the compiled database (`PokemonData`). Each active Pokémon uses a separate in-memory battle instance (`pokemonInBattle`) that tracks changing state such as current HP, PP, status, and stat stages. Since Pokemon games allow users to catch duplicate yet unique Pokemon species, we wanted to mimic that behaviour for our project. Additionally, having `pokemonInBattle` point to a `PokemonData` template makes adding new species, moves, or modifying templates simple, with a data change rather than refactoring battle logic.

9.4 ID Based Sprite Selection

Battle sprites are selected by Pokémon ID and front/back sprites through initialization in `pokemonSpriteInit.c`. Doing so made the graphics code maintainable and simple to add more sprites as the project grew.

10 Future Implementation

10.1 Testing During Development and Future Unit Testing

During development, the game logic was tested and validated with gameplay and integration testing of complete flows (movement and UI navigation, battle actions, status effects, catching/fleeing, and EXP/level ups). We did this to ensure that our project behaved correctly when combined and that messaging and animations remained consistent under our frame dependent model for timing.

For future work, our codebase is suited for adding unit tests because many mechanics are implemented as small, contained functions (eg. `getTypeEffectiveness`, `useAttack`, `dealDamage`, `tickStatusEffect`, `attemptCatchWithBall`, `attemptFlee`). This would be one by separating our game logic code into small executable modules for host-based unit testing. this would allow deterministic combat rules (immunity, STAB, PP, and status conditions) to be tested and verified automatically. For our probability-based mechanics, such as catching, repeated trials would be used with **fixed random seeds**, like seeding `rand()` with a constant so results are reproducible, making randomness testable.

Planned integration tests for this project include checking battle logic across all 4 actions types, 5 status conditions, all 18 type matchup, and all 5 Pokeball types across varying HP states. All the map areas and their transitions will be traversed, battling the 2 trainers to verify payout and party switching logic.

10.2 Saving and Loading Progress

As our project's main menu already has a menu UI for saving, having persistent save data is a natural extension to the project so progression is not lost on rest. One approach is to write and serialize the user's state (party, PC, bag contents, Pokedex, XY and map positions, and progress flags) into a JSON file which would be written and read from storage. On the DE1-SoC, using a microSD would likely achieve similar results. The game would write a `save.json` file when the user saves, and parse it during compilation and boot to restore progression. To detect corruption, this file would also include a parameter for `save_version` for backwards compatibility and backup.

References

1. The Spriters Resource. (2026) Pokemon black and white sprites. [Online]. Available: https://www.spriters-resource.com/ds_dsi/pokemonblackwhite/
2. Pokemon Database. (2026) Pokemon back and front sprites. [Online]. Available: <https://pokemondb.net/sprites>
3. Msikma. (2026) Pokesprite box sprites. [Online]. Available: <https://msikma.github.io/pokesprite/index.html>
4. Atichii. (2026) Title screen artwork. [Online]. Available: <https://cara.app/atichiis>
5. Audreyeyeyeye. (2026) Pokemon ignis/cobalt final pause menu ui. [Online]. Available: <https://www.deviantart.com/audreyeyeyeye/art/Pokemon-Ignis-Cobalt-Final-Pause-Menu-UI-775269868>
6. Reddit. (2026) Forest grass variation background. [Online]. Available: https://www.reddit.com/r/PokemonROMhacks/comments/1n5gx06/forest_grass_variation_makes_a_world_of_difference/
7. Retr0nc. (2026) Pokemon black and white evolution screen. [Online]. Available: <https://www.deviantart.com/retronc/art/Pokemon-Black-White-Evolution-Screen-1066835830>
8. Piskel. (2026) Piskel sprite editor. [Online]. Available: <https://www.piskelapp.com/>
9. Dhepper. (2026) Font8x8 bitmap fonts. [Online]. Available: <https://github.com/dhepper/font8x8>
10. Wikipedia. (2026) Bezier curve. [Online]. Available: https://en.wikipedia.org/wiki/B%C3%A9zier_curve
11. Bulbapedia. (2026) Pokemon damage formula. [Online]. Available: <https://bulbapedia.bulbagarden.net/wiki/Damage>
12. wikimedia. (2026) Pokemon type chart. [Online]. Available: https://commons.wikimedia.org/wiki/File:Pokemon_Type_Chart.svg
13. Bulbapedia. (2026) Pokemon experience formula. [Online]. Available: <https://bulbapedia.bulbagarden.net/wiki/Experience>
14. MyInstants. (2026) Pokemon soundboard. [Online]. Available: <https://www.myinstants.com/en/search/?name=pokemon>
15. KHInsider. (2026) Pokemon ruby & sapphire music super complete. [Online]. Available: <https://downloads.khinsider.com/game-soundtracks/album/pokemon-ruby-sapphire-music-super-complete>
16. Game Development Stack Exchange. (2026) Color key or transparent. [Online]. Available: <https://gamedev.stackexchange.com/questions/103761/color-key-or-transparent-images?>